

1. Войдите под пользователем user1 из практики 2 (su - user1)

```
script started, output log file is /dev/null
root@eltex-practice2-v12:~# su - user1
user1@eltex-practice2-v12:~$
```

2. Подсчитайте количество процессов, имеющих несколько потоков выполнения

```
user1@eltex-practice2-v12:~$ ps -e -L --no-headers | awk '{print $1}' | sort | uniq -c | awk '$1 > 1' | wc -l
10
user1@eltex-practice2-v12:~$
```

3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:

- удалите поля VIRT, RES, SHR;
- добавьте поле RUSER и сделайте так, чтобы это поле было

показано после поля USER;

MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 3444.1 avail Mem											
PID	PPID	TIME+	%CPU	%MEM	USER	RUSER	PR	NI	S	UID	COMMAND
5107	5106	0:00.01	0.0	0.1	user1	user1	20	0	S	1004	bash
5129	5107	0:00.05	0.0	0.2	user1	user1	20	0	R	1004	top
559	1	0:00.21	0.0	0.2	systemd+	systemd+	20	0	S	997	systemd-timesyn
558	1	0:00.19	0.0	0.3	systemd+	systemd+	20	0	S	992	systemd-resolve
548	1	0:00.59	0.0	0.2	systemd+	systemd+	20	0	S	998	systemd-network
713	1	0:00.09	0.0	0.2	syslog	syslog	20	0	S	103	rsyslogd
1	0	0:02.48	0.0	0.3	root	root	20	0	S	0	systemd
2	0	0:00.01	0.0	0.0	root	root	20	0	S	0	kthreadd
3	2	0:00.00	0.0	0.0	root	root	20	0	S	0	pool_workqueue_release
4	2	0:00.00	0.0	0.0	root	root	0	-20	I	0	kworker/R-rcu_g
5	2	0:00.00	0.0	0.0	root	root	0	-20	I	0	kworker/R-rcu_p
6	2	0:00.00	0.0	0.0	root	root	0	-20	I	0	kworker/R-slub
7	2	0:00.00	0.0	0.0	root	root	0	-20	I	0	kworker/R-netns
12	2	0:00.00	0.0	0.0	root	root	0	-20	I	0	kworker/R-mm_pe
13	2	0:00.00	0.0	0.0	root	root	20	0	I	0	rcu_tasks_kthread
14	2	0:00.00	0.0	0.0	root	root	20	0	I	0	rcu_tasks_rude_kthread
15	2	0:00.00	0.0	0.0	root	root	20	0	I	0	rcu_tasks_trace_kthread
16	2	0:00.03	0.0	0.0	root	root	20	0	S	0	ksoftirqd/0
17	2	0:00.85	0.0	0.0	root	root	20	0	I	0	rcu_preempt
18	2	0:00.30	0.0	0.0	root	root	rt	0	S	0	migration/0
19	2	0:00.00	0.0	0.0	root	root	-51	0	S	0	idle_inject/0
20	2	0:00.00	0.0	0.0	root	root	20	0	S	0	cpuhp/0
21	2	0:00.00	0.0	0.0	root	root	20	0	S	0	cpuhp/1
22	2	0:00.00	0.0	0.0	root	root	51	0	S	0	idle_inject/1

4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля

```
user1@eltex-practice2-v12: ~
[screen is terminating]
user1@eltex-practice2-v12:~$ exit
logout
root@eltex-practice2-v12:~# su - user1
user1@eltex-practice2-v12:~$ passwd
Changing password for user1.
Current password:
```

5. Перейдите в терминальное окно с top и выполните следующие действия:

- выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;
- найдите процесс, запущенный командой passwd;
- отправьте этому процессу сигналы 15 (SIGTERM), 2 (SIGINT), 3 (SIGQUIT), 9(SIGKILL)

root@eltex-practice2-v12: ~

```
top - 08:07:21 up 23:23, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 113 total, 1 running, 112 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3915.9 total, 2797.8 free, 472.3 used, 903.8 buff/cache
MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 3443.6 avail Mem
Which user (blank for all) user1

  PID USER      PR  NT   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 5176 root        0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
 5177 root        0.0  0.1   9400   4724 pts/2  Ss+  08:05    0:00 su - user1 -c passwd
 5178 root        0.0  0.0   9172   3892 ?     Ss   08:05    0:00 passwd
 5184 root        0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~# ps aux | grep passwd | grep -v grep
root      5176  0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
root      5177  0.0  0.1   9400   4724 pts/2  Ss+  08:05    0:00 su - user1 -c passwd
root      5178  0.0  0.0   9172   3892 ?     Ss   08:05    0:00 passwd
root      5184  0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~# kill -15 5178

root@eltex-practice2-v12:~# ps aux | grep passwd | grep -v grep
root      5176  0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
root      5177  0.0  0.1   9400   4724 pts/2  Ss+  08:05    0:00 su - user1 -c passwd
root      5178  0.0  0.0   9172   3892 ?     Ss   08:05    0:00 passwd
root      5184  0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~# kill -2 5178

root@eltex-practice2-v12:~# ps aux | grep passwd | grep -v grep
root      5176  0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
root      5177  0.0  0.1   9400   4724 pts/2  Ss+  08:05    0:00 su - user1 -c passwd
root      5178  0.0  0.0   9172   3892 ?     Ss   08:05    0:00 passwd
root      5184  0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~# kill -3 5178

ps aux | grep passwd | grep -v grep
root      5176  0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
root      5177  0.0  0.1   9400   4724 pts/2  Ss+  08:05    0:00 su - user1 -c passwd
root      5178  0.0  0.0   9172   3892 ?     Ss   08:05    0:00 passwd
root      5184  0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~# kill -9 5178

ps aux | grep passwd | grep -v grep
root      5176  0.0  0.0   7604   3124 ?     Ss   08:05    0:00 SCREEN -S passwd_session -d -m su - user1 -c passwd
root      5184  0.0  0.0   6940   3304 pts/1  S+   08:05    0:00 screen -r passwd_session

root@eltex-practice2-v12:~#
```

6. Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z

root@eltex-practice2-v12: ~

```
Hello World!
```

```
~
~
~
~
~
~
```

```
root@eltex-practice2-v12:~# jobs
[1]+  Stopped                  vim ~/file_task3.txt
root@eltex-practice2-v12:~#
```

7. Выполните команду `sleep 600`, нажмите `Ctrl-Z` и выполните команду `jobs`

```
[1]+  Stopped                  vim ~/file_task3.txt
root@eltex-practice2-v12:~# jobs
[1]+  Stopped                  vim ~/file_task3.txt
root@eltex-practice2-v12:~# sleep 600
^Z
[2]+  Stopped                  sleep 600
root@eltex-practice2-v12:~# jobs
[1]-  Stopped                  vim ~/file_task3.txt
[2]+  Stopped                  sleep 600
root@eltex-practice2-v12:~#
```

8. Последнее задание (`sleep 600`) сделайте фоновым

```
root@eltex-practice2-v12:~# bg %2
[2]+ sleep 600 &
root@eltex-practice2-v12:~# jobs
[1]+  Stopped                  vim ~/file_task3.txt
[2]-  Running                  sleep 600 &
root@eltex-practice2-v12:~#
```

9. Измените число NICE у задания (`sleep 600`), сделав его равным 10

```
root@eltex-practice2-v12:~# jobs -l
[1]+  5271 Stopped                  vim ~/file_task3.txt
[2]-  5273 Running                  sleep 600 &
root@eltex-practice2-v12:~# renice -n 10 5273
5273 (process ID) old priority 0, new priority 10
root@eltex-practice2-v12:~#
```

10. Проверьте, что число NICE у этого задания изменилось

```
root@eltex-practice2-v12:~# ps -l 5273
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   0       5273     5185  0  90   10 -   1421 do_sys pts/3      0:00 sleep 600
root@eltex-practice2-v12:~#
```

11. Сделайте задание `vim ~/file_task3.txt` активным и выйдите из редактора

`fg %1` и нажимаем `Esc`, пишем `:q!`

12. Отправьте сигнал 15 (SIGTERM) заданию sleep 600 и выполните команду jobs

```
root@eltex-practice2-v12:~# kill -15 5273
jobs
[2]+  Running                  sleep 600 &
root@eltex-practice2-v12:~#
```

13. Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду trap) и отправьте сигналы самому себе

```
root@eltex-practice2-v12:~# trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
root@eltex-practice2-v12:~# kill -INT $$
Меня голыми руками не возьмёшь!
root@eltex-practice2-v12:~# kill -QUIT $$
Меня голыми руками не возьмёшь!
root@eltex-practice2-v12:~#
```

Раздел 2

1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:

- Создает именованный канал с именем cuckoo.pid в каталоге /tmp/run/
- Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
- В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?» возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
- При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу.

```
root@eltex-practice2-v12:~# rm -f /root/cuckoo.sh
root@eltex-practice2-v12:~# cat > /root/cuckoo.sh << 'EOF'
> #!/bin/bash
PIPE="/tmp/run/cuckoo.pid"
LOG="/tmp/cuckoo.log"
mkdir -p /tmp/run
rm -f $PIPE
mkfifo $PIPE
echo "$(date '+%Y-%m-%d %H:%M:%S') Startup!" >> $LOG
trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> $LOG; rm -f $PIPE; exit 0' SIGTERM
while true; do
    if read -r request < $PIPE; then
        if [[ $request =~ ^([A-Za-z0-9_]+)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\?$ ]]; then
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"
            N=$((RANDOM % 9 + 2))
            echo "$N" > $PIPE
            echo "$(date '+%Y-%m-%d %H:%M:%S') $NAME[$PID] $N" >> $LOG
        else
            echo "$(date '+%Y-%m-%d %H:%M:%S') Unknown request: $request" >> $LOG
        fi
    fi
done
EOF
root@eltex-practice2-v12:~# chmod +x /root/cuckoo.sh
root@eltex-practice2-v12:~# cat /root/cuckoo.sh
#!/bin/bash
PIPE="/tmp/run/cuckoo.pid"
LOG="/tmp/cuckoo.log"
mkdir -p /tmp/run
rm -f $PIPE
mkfifo $PIPE
echo "$(date '+%Y-%m-%d %H:%M:%S') Startup!" >> $LOG
trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> $LOG; rm -f $PIPE; exit 0' SIGTERM
while true; do
    if read -r request < $PIPE; then
        if [[ $request =~ ^([A-Za-z0-9_]+)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\?$ ]]; then
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"
            N=$((RANDOM % 9 + 2))
            echo "$N" > $PIPE
            echo "$(date '+%Y-%m-%d %H:%M:%S') $NAME[$PID] $N" >> $LOG
        else
            echo "$(date '+%Y-%m-%d %H:%M:%S') Unknown request: $request" >> $LOG
        fi
    fi
done
root@eltex-practice2-v12:~# █
```

2. Запустите скрипт cuckoo.sh при помощи nohup или screen

```
done
root@eltex-practice2-v12:~# nohup /root/cuckoo.sh &
[1] 5315
root@eltex-practice2-v12:~# nohup: ignoring input and appending output to 'nohup.out'
ps aux | grep cuckoo.sh | gr^C
root@eltex-practice2-v12:~# ps aux | grep cuckoo.sh | grep -v grep
root      5315  0.0  0.0   7340  3656 pts/2    S    08:47   0:00 /bin/bash /root/cuckoo.sh
root@eltex-practice2-v12:~# ls -la /tmp/run/cuckoo.pid
prw-r--r-- 1 root root 0 Aug 13 08:47 /tmp/run/cuckoo.pid
root@eltex-practice2-v12:~# cat /tmp/cuckoo.log
2026-08-13 08:47:50 Startup!
root@eltex-practice2-v12:~#
```

3. Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:

- При запуске проверяет, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
- При запуске дописывает в файл report_имя_скрипта_без_полного_пути.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
- Запрашивает через именованный канал скрипта cuckoo время сообщением в указанном ранее формате и ждет такое количество секунд.
- Дописывает в файл report_имя_скрипта_без_полного_пути.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.

```
root@eltex-practice2-v12:~# cat > /root/template_task.sh << 'EOF'
> #!/bin/bash

SCRIPT_NAME=$(basename "$0")
if [[ "$SCRIPT_NAME" == "template_task.sh" ]]; then
    echo "я бригадир, сам не работаю"
    exit 1
fi

PIPE="/tmp/run/cuckoo.pid"
LOG_FILE="report_${SCRIPT_NAME%.sh}.log"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт запущен" >> "$LOG_FILE"

REQUEST="${SCRIPT_NAME}[$$]: how much time do I have?"
echo "$REQUEST" > "$PIPE"
read -r N < "$PIPE"

sleep "$N"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт завершился, работал $N секунд" >> "$LOG_FILE"
EOF
root@eltex-practice2-v12:~# chmod +x /root/template_task.sh
root@eltex-practice2-v12:~# cat /root/template_task.sh
#!/bin/bash

SCRIPT_NAME=$(basename "$0")
if [[ "$SCRIPT_NAME" == "template_task.sh" ]]; then
    echo "я бригадир, сам не работаю"
    exit 1
fi

PIPE="/tmp/run/cuckoo.pid"
LOG_FILE="report_${SCRIPT_NAME%.sh}.log"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт запущен" >> "$LOG_FILE"

REQUEST="${SCRIPT_NAME}[$$]: how much time do I have?"
echo "$REQUEST" > "$PIPE"
read -r N < "$PIPE"

sleep "$N"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт завершился, работал $N секунд" >> "$LOG_FILE"
root@eltex-practice2-v12:~#
```

4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log

```
done < $(date '+%Y-%m-%d %H:%M:%S') Скрипт observer.sh работает в секунду >> $LOG_FILE
root@eltex-practice2-v12:~# cat > /root/observer.sh << 'EOF'
> #!/bin/bash

CONFIG="/root/observer.conf"
LOG_FILE="/root/observer.log"

if [[ ! -f "$CONFIG" ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') Файл конфигурации $CONFIG не найден" >> "$LOG_FILE"
    exit 1
fi

while IFS= read -r script_path; do
    [[ -z "$script_path" ]] && continue
    SCRIPT_NAME=$(basename "$script_path")
    if ! pgrep -f "$SCRIPT_NAME" > /dev/null; then
        nohup "$script_path" &
        echo "$(date '+%Y-%m-%d %H:%M:%S') Перезапущен $SCRIPT_NAME" >> "$LOG_FILE"
    fi
done < "$CONFIG"
EOF
root@eltex-practice2-v12:~# chmod +x /root/observer.sh
root@eltex-practice2-v12:~# cat /root/observer.sh
#!/bin/bash

CONFIG="/root/observer.conf"
LOG_FILE="/root/observer.log"

if [[ ! -f "$CONFIG" ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') Файл конфигурации $CONFIG не найден" >> "$LOG_FILE"
    exit 1
fi

while IFS= read -r script_path; do
    [[ -z "$script_path" ]] && continue
    SCRIPT_NAME=$(basename "$script_path")
    if ! pgrep -f "$SCRIPT_NAME" > /dev/null; then
        nohup "$script_path" &
        echo "$(date '+%Y-%m-%d %H:%M:%S') Перезапущен $SCRIPT_NAME" >> "$LOG_FILE"
    fi
done < "$CONFIG"
root@eltex-practice2-v12:~#
```

5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```
/root/template_task.sh
root@eltex-practice2-v12:~# (crontab -l 2>/dev/null; echo "* * * * * /root/observer.sh") | crontab -
root@eltex-practice2-v12:~# crontab -l
* * * * * /root/observer.sh
root@eltex-practice2-v12:~#
```

6. Создайте несколько символических ссылок на файл `template_task.sh` с различными именами (рабочие задачи), добавьте в файл конфигурации `observer.conf` соответствующие записи об этих задачах, включая исходный файл `template_task.sh`

```
root@eltex-practice2-v12:~# cd /root
root@eltex-practice2-v12:~# ln -s template_task.sh task1.sh
root@eltex-practice2-v12:~# ln -s template_task.sh task2.sh
root@eltex-practice2-v12:~# ln -s template_task.sh task3.sh
root@eltex-practice2-v12:~# ls -la /root/task*.sh
lrwxrwxrwx 1 root root 16 Aug 13 08:52 /root/task1.sh -> template_task.sh
lrwxrwxrwx 1 root root 16 Aug 13 08:52 /root/task2.sh -> template_task.sh
lrwxrwxrwx 1 root root 16 Aug 13 08:52 /root/task3.sh -> template_task.sh
root@eltex-practice2-v12:~# cat > /root/observer.conf << EOF
> /root/task1.sh
/root/task2.sh
/root/task3.sh
/root/template_task.sh
EOF
root@eltex-practice2-v12:~# cat /root/observer.conf
/root/task1.sh
/root/task2.sh
/root/task3.sh
/root/template_task.sh
root@eltex-practice2-v12:~#
```

7. Соберите статистику работы в виде набора файлов `report_*.log`, `observer.log`, `сискоо.log` приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива `tar`. Не забудьте остановить процесс, удалив задачу в `cron`!